COMP1511
Introduction to Discrete Mathematics

Lecture Notes
2023/24

The three main topics that are covered in this module are:

1. Combinatorics: Counting Methods
2. Discrete Probability Theory
3. Graph Theory

The lecture notes closely follow the material presented in Chapters 6, 7, 10 and 11 of the accompanying book (that is referred to in the rest of the text as *the book*):

> K. H. Rosen, **Discrete Mathematics and its Applications**, 7th Edition, McGraw Hill Education, 2013.

All of the material from the module syllabus will be covered in the lecture notes and presented in the lectures. The examination and coursework will only assess topics covered in the module syllabus. You should use the book as an additional source of problems to solve and as an alternative explanation of a topic. The book is there to supplement the lecture notes. Not all of the material from the chapters is covered in the lecture notes (i.e. lectures), and the lectures sometimes cover the material in a different order than the one in the book.

We expect you, as a student, to:

1. Attend lectures and carefully read the lecture notes.
2. Attend and engage in tutorials, completing all problems requested by your tutorial tutor.
3. Submit all courseworks.
4. To truly absorb the material covered, you should also practice exercises from the relevant sections of the book (the odd numbered ones have solutions in the back).

1 Combinatorics: Counting Methods (Corresponds to Sections 6.1 - 6.5 in the book)

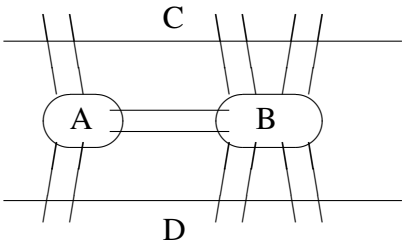
2 Discrete Probability Theory (Corresponds to Sections 7.1 - 7.3 in the book)

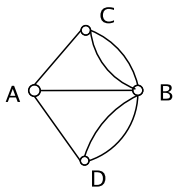
3 Graph Theory (Corresponds to selected material from Chapters 10 and 11 in the book)

3.1 Introduction

The first paper in graph theory goes back to 1736, and some important results were obtained in the nineteenth century. Since the 1920s graph theory started developing at great speed, due to its modern applicability in diverse fields such as computer science, chemistry, operations research, electrical engineering, linguistics and economics.

Königsberg Bridge Problem
(Solved by Euler in 1736)
Two islands lying in the Pregel River in Königsberg, were connected to each other and the river banks by 7 bridges. The mayor of the town wanted to know whether it is possible for a parade to cross each of the seven bridges once and only once during the walk through the town.





Euler proved that this is not possible by proving the following more general result: a connected graph with at least one edge has an Euler cycle if and only if it has no vertices of odd degree.

Friendship Puzzle
At every party there are two people that have exactly the same number of friends at the party.

The Four Color Problem (First appeared in London 1852)
Is it possible to color every map with 4 colors so that no two adjacent countries are colored the same?

In 1879 Kempe published a "solution" and was elected a Fellow of the Royal Society.
In 1890 Heawood published a refutation.
Kempe's idea was that of alternating paths in the Five Color Problem and it led eventually to a proof by Appel and Haken 1976-86. Their proof makes an extensive use of a computer. No proof of this result without the use of a computer is known.

Exam Schedules
Let the vertices of a graph correspond to university courses, and put edges between those courses that have common students.
Then the notion of a chromatic number of a graph, in this example corresponds to the minimum number of time periods needed to schedule examinations without conflicts.

Marriage Theorem
If every girl in a village knows exactly k boys and every boy knows exactly k girls, then each girl can marry a boy she knows and each boy can marry a girl he knows.

Some more examples:

• How many layers does a computer chip need so that wires in the same layer do not cross?

• In what order should a travelling salesman visit cities to minimize travel time?

• What is the maximum flow per unit time from source to sink in a network of pipes?

• How can we lay cable at minimum cost to make every telephone reachable from every other?

3.2 Graph Models

A *simple graph* $G = (V(G), E(G))$ with p vertices and q edges consists of a *vertex set* (or a *node set*) $V(G) = \{v_1, \ldots, v_p\}$ and an *edge set* $E(G) = \{e_1, \ldots, e_q\}$ where each edge is an unordered pair of vertices.

An edge $e = \{u, v\}$ is also denoted by uv or vu.

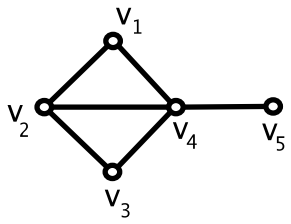
The vertices contained in an edge e are its *endpoints* (or *endnodes* or *endvertices*), and e is said to *connect* u and v.

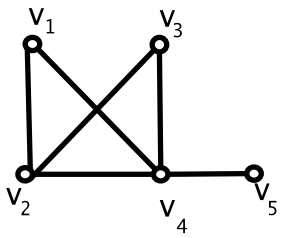
An edge $e = uv$ is said to be *incident* to its endpoints u and v.

Two vertices that are endpoints of the same edge are said to be *adjacent vertices*, and two edges that are incident to the same vertex are said to be *adjacent edges*.

Two adjacent vertices are also called *neighboring vertices*.

Example
$G = (V(G), E(G))$
$V(G) = \{v_1, v_2, v_3, v_4, v_5\}$
$E(G) = \{v_1v_2, v_2v_3, v_3v_4, v_1v_4, v_2v_4, v_4v_5\}$

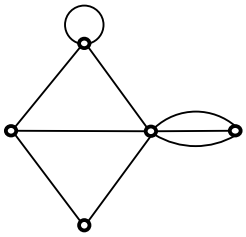




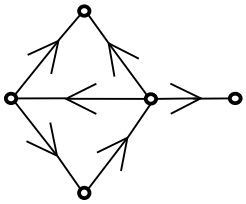
v_1 is adjacent to v_2 and v_4, but not adjacent to v_3 and v_5
v_2 and v_4 are neighbors of v_1

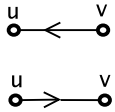
Some other graph models:

• *Multigraphs* (or *graphs* or *undirected graphs*) allow *loops* (edges joining a vertex to itself) and *parallel edges* (several edges joining the same two vertices).
(Note: the terminology introduced so far for simple graphs extends to multigraphs).



• *Directed graphs* (or *digraphs*): edges are ordered pairs of vertices, and are called *directed edges* or *arcs*.

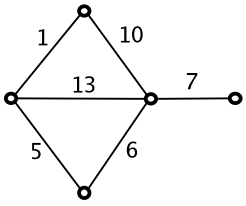




directed edge vu

directed edge uv

• *Weighted graphs*: each edge e is assigned weight $w(e)$



• We can also have different combinations of the above models, such as directed weighted graphs or weighted simple graphs.

Examples of modelling using graphs

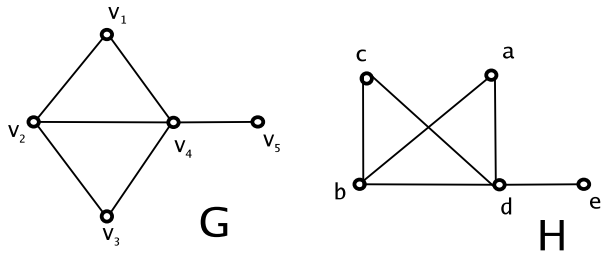
1. A **communication network**: collection of cities such that certain pairs of cities can communicate with each other and others cannot.

2. A **city map**.

3. A **driving time map**: a collection of cities, some of which are joined directly by a highway.

4. A **transportation network**: a network of shipping routes for transporting a commodity from its source to a large processing plant.

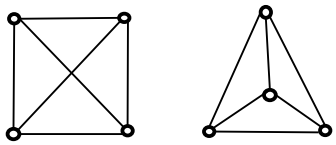
3.3 Isomorphic Graphs



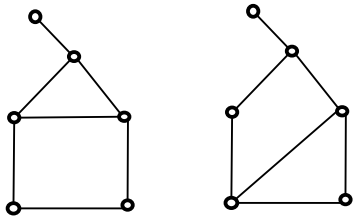
Graphs G and H look exactly the same with the exception that their vertices and edges have different labels. G and H are not identical, but isomorphic.

Let G and H be simple graphs. G is *isomorphic* to H, denoted by $G \cong H$, if there exists a bijection $f : V(G) \to V(H)$ such that $uv \in E(G)$ if and only if $f(u)f(v) \in E(H)$.
Graph isomorphism for other graph models is defined similarly.

The following bijection shows that the above graphs G and H are isomorphic: $f(v_1) = a, f(v_2) = b, f(v_3) = c, f(v_4) = d, f(v_5) = e$.



Isomorphic?



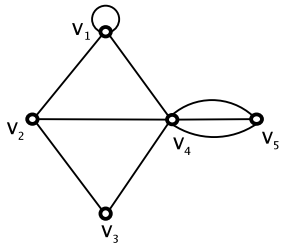
Isomorphic?

Graph isomorphism problem: determine in polynomial time whether two graphs are isomorphic. (This is a very difficult problem that is still unsolved).

Since we will mostly be interested in structural properties of graphs, we will often omit labels when drawing graphs. An unlabelled graph can be thought of as a representative of an equivalence class of isomorphic graphs. We assign labels to vertices and edges in a graph for the purpose of referring to them.

3.4 Vertex Degree

The *degree of a vertex v* in a graph G, denoted by $d_G(v)$ or just $d(v)$ (or sometimes $deg(v)$, as in the book), is the number of edges that are incident to v (with loops counted twice, i.e. each loop on v contributes 2 to $d(v)$).



$d(v_1) =$
$d(v_2) =$
$d(v_3) =$
$d(v_4) =$
$d(v_5) =$

Theorem 3.1 Degree-sum Formula
If G is a graph with vertices $v_1, v_2, \ldots v_n$, then $\sum_{i=1}^{n} d(v_i) = 2|E(G)|$.

Corollary 3.1 *In any graph, the number of vertices of odd degree is even.*

Friendship Puzzle
At any party of two or more people, show that there are always two people with exactly the same number of friends at the party.

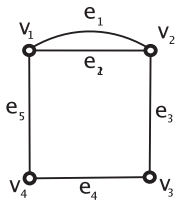
3.5 Paths and Cycles

Caution: In literature there is a considerable variation of terminology concerning the concepts defined in this section. When consulting a book or an article, you need to make sure which definitions are used in the particular text.

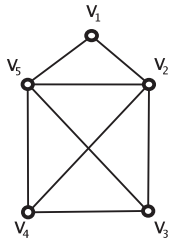
Let v_0 and v_n be vertices in a graph G. A *path* in G from v_0 to v_n of *length* n is an alternating sequence of $n+1$ vertices and n edges, $v_0e_1v_1e_2\ldots v_{n-1}e_nv_n$, in which edge e_i is incident on vertices v_{i-1} and v_i for every $i = 1,\ldots,n$.

Note that in a simple graph, to denote a path we just need to list the sequence of vertices since two vertices are connected by only one edge, e.g. $v_0v_1\ldots v_n$.

A *simple path* from vertex u to vertex v in a graph G is a path from u to v with no repeated vertices.



simple path of length 2: $v_1e_1v_2e_3v_3$
simple path of length 2: $v_1e_2v_2e_3v_3$

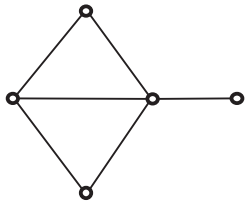


path of length 4: $v_1v_2v_3v_5v_2$
simple path of length 3: $v_1v_2v_4v_5$

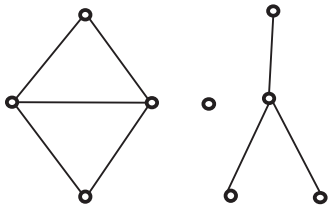
Lemma 3.1 *Let G be a graph, and u and v two vertices of G. There is a path from u to v in G if and only if there is a simple path from u to v in G.*

Two vertices u and v of a graph G are said to be *connected* if there is a path from u to v.

A graph G is *connected* if for every pair of vertices u and v of G there is a path from u to v.
Otherwise G is *disconnected*.



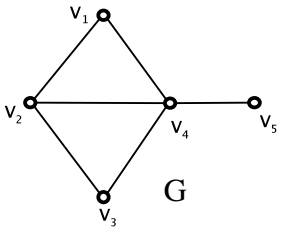
connected graph



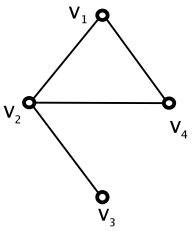
disconnected graph

A graph H is a *subgraph* of a graph G, denoted by $H \subseteq G$, if $V(H) \subseteq V(G)$ and $E(H) \subseteq E(G)$.
A subgraph H of G is a *proper subgraph* of G if $H \neq G$.

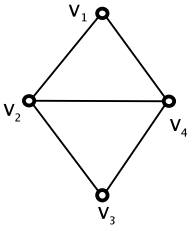
A graph H is an *induced subgraph* of a graph G, if H is a subgraph of G such that every edge of G with both endnodes in $V(H)$ is also an edge of H.
We say that H is the subgraph of G induced by the node set $V(H)$.



G



subgraph of G



induced subgraph of G

Let G be a graph.

For $M \subseteq E(G)$, $G \setminus M$ denotes the graph obtained from G by removing the edges from M. So $G \setminus M = (V(G), E(G) \setminus M)$.

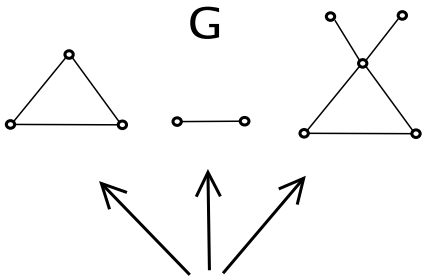
For $S \subseteq V(G)$, $G[S]$ denotes the subgraph of G induced by S.

For $S \subseteq V(G)$, $G \setminus S$ denotes the graph obtained from G by removing the vertices from S (and therefore we also remove all edges of $E(G)$ that are incident to a vertex in S).

So $G \setminus S = G[V(G) \setminus S]$.

Let G be a graph. Relation "is connected to" is an equivalence relation on $V(G)$. It is reflexive, since any vertex is trivially connected to itself by a path of length 0. It is symmetric, since a path from u to v can be traversed in the opposite direction to give a path from v to u. It is transitive, since a path from u to v and a path from v to w can be combined to give a path from u to w.

So there is a partition of $V(G)$ into nonempty subsets $V_1, \ldots, V_k$ such that u and v are connected if and only if both u and v belong to the same subset V_i. For $i = 1, \ldots, k$, we define G_i to be the subgraph of G induced by the node set V_i. Induced subgraphs $G_1, \ldots, G_k$ are called the *components* of G. Note that connected graphs are those that have only one component.



components of G

Another way of defining components of G is as follows. A *component* (or *connected component*) of G is a maximal (in terms of the number of nodes and edges) connected subgraph of G. In other words, a connected subgraph C of G is a component of G if there does not exist a connected subgraph C' of G such that $V(C) \subseteq V(C')$, $E(C) \subseteq E(C')$ and $|V(C) \cup E(C)| < |V(C') \cup E(C')|$.

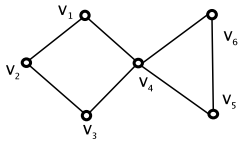
Let G be a graph and v a vertex of G.

A *cycle* is a path of nonzero length from v to v with no repeated edges.

The *length* of a cycle is the number of edges it contains.

A *simple cycle* is a cycle from v to v in which, except for the beginning and ending vertices that are both equal to v, there are no repeated vertices.

(Note that the definition of a cycle in the book coincides with our definition of a simple cycle, and the definition of a simple circuit in the book coincides with our definition of a cycle).



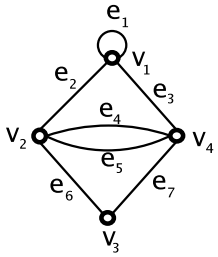
cycle: $v_1v_2v_3v_4v_5v_6v_4v_1$

simple cycle: $v_1v_2v_3v_4v_1$

Euler Cycles

A cycle in a graph G that includes all of the edges and all of the vertices of G is called an *Euler cycle* (or an *Euler tour*, and in the book it is an Euler circuit).

A graph is *Eulerian* if it has an Euler cycle.



Euler cycle:

$v_1e_1v_1e_2v_2e_6v_3e_7v_4e_4v_2e_5v_4e_3v_1$

Theorem 3.2 Euler's Theorem
A connected graph with at least one edge has an Euler cycle if and only if it has no vertices of odd degree.

Recognition algorithm for Eulerian graphs

Input: A graph G.

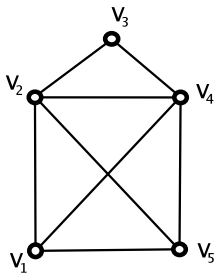
Output: YES if G is Eulerian and NO otherwise.

Step 1: Check whether G is connected (using for example Breadth First Search or Depth First Search algorithms).
If it is, then continue; otherwise return NO and stop.

Step 2: Check whether the degree of every vertex of G is even.
If it is, then return YES; otherwise return NO.

Correctness of this algorithm follows from Euler's Theorem. By following the given proof of Euler's Theorem, one can extract an algorithm that finds an Euler cycle if one exists.

In a graph G an *Euler path* is a path that includes all of the vertices of G, traverses every edge of G exactly once and it begins and ends at distinct vertices.



Euler path:

$v_1v_2v_3v_4v_2v_5v_4v_1v_5$

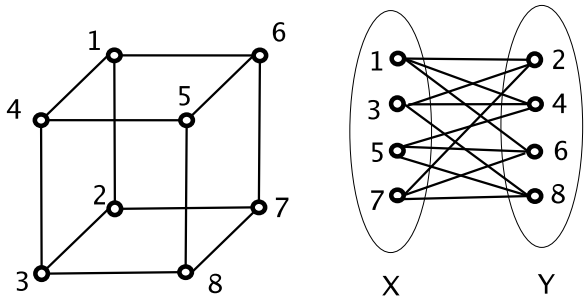
Corollary 3.2 *A connected graph G has an Euler path if and only if it has exactly two vertices of odd degree.*

Bipartite Graphs

A *bipartite graph* is a graph G whose vertices can be partitioned into two (possibly empty) subsets X and Y (so $X \cap Y = \emptyset$ and $X \cup Y = V(G)$) such that each edge of G has one end in X and one in Y.

(X,Y) is called a *bipartition* of G.

Note that a simple graph G on one vertex, say u, is bipartite since we can let $X = \{u\}$ and $Y = \emptyset$. Another example of a bipartite graph is given in the figure.



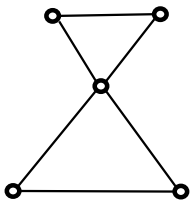
bipartite graph

We now characterize bipartite graphs. A *closed path* is a path that begins and ends at the same vertex. We say that a path or a cycle is *even* or *odd* depending on the parity of its length.

Note that the graph $G = (\{u,v\}, \{\{u,v\}\})$ has a closed path of positive length (e.g. u,v,u,v,u is a closed path of G of length 4), but does not have a cycle.

Lemma 3.2 *Every odd closed path contains an odd simple cycle.*

Note that it is not true that every even closed path contains an even simple cycle.



Theorem 3.3 *A graph is bipartite if and only if it contains no odd simple cycle.*

Recognition algorithm for bipartite graphs

Input: A connected graph G.

Output: YES if G is bipartite (together with its bipartition), and NO otherwise.

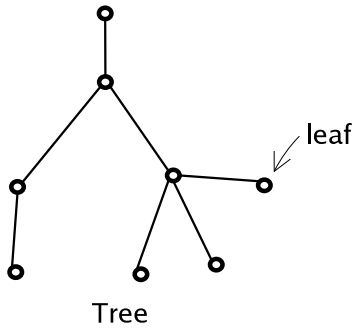
Step 1: For some vertex $u \in V(G)$, search the graph from u (using for example Breadth First Search or Depth First Search algorithms).
As vertices are discovered partition them into sets X and Y as follows: for $v \in V(G)$, if v is reached on an even path, then put v in X, and otherwise put v in Y.

Step 2: Check whether G has an edge ab such that $\{a,b\} \subseteq X$ or $\{a,b\} \subseteq Y$. If no such edge exists, then return YES and bipartition (X,Y); otherwise return NO.

3.6 Trees

A *tree* is a connected graph with no cycles.

A *leaf* is a vertex of degree 1.



Lemma 3.3 *Every tree with at least two vertices has at least two leaves.*

Note that deleting a leaf from a tree with n vertices, results in a tree with $n-1$ vertices.

In a connected graph G, an edge e is a *cut-edge* if the graph obtained from G by removing e is disconnected.

Lemma 3.4 *Let G be a connected graph. An edge e is not a cut-edge if and only if it belongs to a cycle.*

Theorem 3.4 *For a simple graph G with n vertices, $n \geq 1$, the following are equivalent:*

(a) G is connected and has no cycles.

(b) G is connected and has $n-1$ edges.

(c) G has $n-1$ edges and no cycles.

(d) For every pair $u, v \in V(G)$, G has exactly one simple path from u to v.